**Institución Universitaria Pascual Bravo**
**Facultad de Ingeniería - Departamento de Sistemas Digitales**
**Base de Datos I (SD1006)**

**Experiencia de Aprendizaje (EA4) - Proyecto de Aula (Fase 4)**

# Implementación de la estructura completa de una base de datos (DDL) en tres (3) SGBDs diferentes

- **Tarea en Equipo**
- **Peso: 15%**
- **Práctica. Caso de Estudio**
- **Modelo Físico - Diccionario de Datos en DBMS - PostgreSQL, MySQL, MS SQL Server**
- **Datos Estructurados y Semi Estructurados**


**MIEMBROS DEL EQUIPO:**

- Líder: Samuel Perez Jaramillo
- Miembro: Tomas Acuña Alzate
- Miembro:Santiago Mejia Torres
- Miembro:Samuel Moreno Salazar

**Institución Universitaria Pascual Bravo**
**Facultad de Ingeniería - Departamento de Sistemas Digitales**
**Base de Datos I (SD1006)**

# Informe con resultados

## 1.- Tabla Comparativa de tipos de Datos

| Tipos de Datos - Tabla Comparativa | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| # | Tipo de Dato Genérico | Descripción | En PosgreSQL Version 18 | | | En MySQL Version 8.0.46 | | | En MS SQL Server Version 17.0.1000.7 | | |
| | | | Tipo de dato | Dominio | Tamaño Bytes | Tipo de dato | Dominio | Tamaño Bytes | Tipo de dato | Dominio | Tamaño Bytes |
| 1 | AUTOINCREMENTABLE | Genera automáticamente valores consecutivos. | SERIAL | Según el tipo numérico (TINYINT a BIGINT) | 4 BYTE S | AUTO_INCREMENT | Según el tipo numérico (TINYINT a BIGINT) | DEPENDE DEL TIPO | IDENTITY | Según el tipo numérico (TINYINT a BIGINT) | DEPENDE DEL TIPO |
| 2 | UUID | Identificador único de 128 bits. | UUID | Cadenas de 128 bits / 32 dígitos hex en formato 8-4-4-4-12 | 16 BYTE S | CHAR(36) | Cadenas de 36 caracteres A SCII (formato UUID) | 36 BYTE S | UNIQUEIDENTIFIER | adenas de 128 bits / 32 dígitos hex en formato 8-4-4-4- | 16 BYTE S |
| 3 | ENTERO | Números enteros sin decimales. | INTEGER | -2,147,483,648 a 2,147,483,647 | 4 BYTE S | INT | -2,147,483,648 a 2,147,483,647 (Signed) | 4 BYTE S | INT | -2,147,483,648 a 2,147,483,647 | 4 BYTE S |
| 4 | DECIMAL | Números con decimales y precisión definida. | NUMERIC | Hasta 131,072 dígitos antes del decimal y 16,383 después | VARIABLE | DECIMAL | Hasta 65 dígitos | VARIABLE | DECIMAL | Hasta 38 dígitos | VARIABLE |
| 5 | LÓGICO | Representa verdadero o falso. | BOOLEAN | true, false, null | 1 BYTE | BOOLEAN | 1, 0, null (alias de TINYINT(1)) | 1 BYTE | BIT | 1, 0, null | 1 BYTE |
| 6 | TEXTO | Almacena texto de longitud variable. | TEXT | Cadena de texto de longitud ilimitada (hasta 1 GB) | VARIABLE | TEXT | Cadena de texto de hasta 65,535 bytes (~65 KB) | VARIABLE | VARCHAR(MAX) | Cadena de caracteres de hasta 2 GB (VARCHAR(MAX)) | VARIABLE |
| 7 | CARACTER | Almacena caracteres de longitud fija. | CHAR | 1 carácter | 1 BYTE | CHAR | Hasta 255 caracteres | VARIABLE | CHAR | Hasta 8,000 caracteres | VARIABLE |
| 8 | FECHA | Almacena una fecha sin hora. | DATE | 4713 a.C. a 5874897 d.C. (YYYY-MM-DD) | 4 BYTE S | DATE | 1000-01-01 a 9999-12-31 (YYYY-MM-DD) | 3 BYTE S | DATE | 0001-01-01 a 9999-12-31 (YYYY-MM-DD) | 3 BYTE S |
| 9 | FECHA_HORA | Almacena fecha y hora. | TIMESTAMP | 4713 a.C. a 294278 d.C. | 8 BYTE S | DATETIME | 1000-01-01 00:00:00 a 9999-12-31 23:59:59 | 8 BYTE S | DATETIME2 | 0001-01-01 a 9999-12-31 (precisión de 100 ns) | 6–8 BYTE S |
| 10 | HORA | Almacena solamente la hora. | TIME | 00:00:00 a 24:00:00 | 8 BYTE S | TIME | -838:59:59 a 838:59:59 | 3–8 BYTE S | TIME | 00:00:00.0000000 a 23:59:59.9999999 | 3–5 BYTE S |
| 11 | URL | Almacena direcciones web. | VARCHAR | Cadena de texto variable (hasta 1 GB) | VARIABLE | VARCHAR | Cadena de texto variable (hasta 65,535 bytes) | VARIABLE | VARCHAR | Cadena de caracteres de hasta 8,000 caracteres | VARIABLE |
| 12 | ENUM | Permite valores de una lista definida. | ENUM | Lista estática de elementos definidos por el usuario | 4 BYTE S | ENUM | Lista estática de hasta 65,535 elementos | 1–2 BYTE S | VARCHAR + CHECK | Definido por restricción CHECK (col IN ('A', 'B')) | VARIABLE |
| 13 | TEXTO_CORTO | Almacena texto de longitud limitada. | VARCHAR | Cadena de texto variable (hasta 1 GB o limitado por n) | VARIABLE | VARCHAR | Cadena de texto de 1 a 65,535 caracteres | VARIABLE | VARCHAR | Cadena de caracteres de 1 a 8,000 caracteres | VARIABLE |
| 14 | BINARIO | Almacena datos binarios. | BYTEA | Datos binarios de longitud variable (hasta 1 GB) | VARIABLE | BLOB | Datos binarios de hasta 65,535 bytes (BLOB) | VARIABLE | VARBINARY(MAX) | Datos binarios de hasta 2 GB (VARBINARY(MAX)) | VARIABLE |
| 15 | JSON | Almacena datos en formato JSON. | JSON | Documento con sintaxis JSON válida | VARIABLE | JSON | Documento con sintaxis JSON válida | VARIABLE | NVARCHAR(MAX) | to con sintaxis JSON válida (almacenado en NVARCHA | VARIABLE |
| 16 | JSONB | JSON en formato binario optimizado. | JSONB | ocumento JSON válido descompuesto (binario optimizad | VARIABLE | JSON | Documento JSON válido optimizado en binario | VARIABLE | NVARCHAR(MAX) | umento JSON válido (almacenado en NVARCHAR(MA | VARIABLE |
| 17 | IMAGEN | Almacena imágenes como datos binarios. | BYTEA | Datos binarios de imágenes (hasta 1 GB) | VARIABLE | BLOB | os binarios de imágenes (hasta 65,535 bytes o LONGBLOB de 4 | VARIABLE | VARBINARY(MAX) | Datos binarios de imágenes (hasta 2 GB) | VARIABLE |
| 18 | AUDIO | Almacena audio como datos binarios. | BYTEA | Datos binarios de audio (hasta 1 GB) | VARIABLE | BLOB | tos binarios de audio (hasta 65,535 bytes o LONGBLOB de 4 GB | VARIABLE | VARBINARY(MAX) | Datos binarios de audio (hasta 2 GB) | VARIABLE |

**Institución Universitaria Pascual Bravo**
**Facultad de Ingeniería - Departamento de Sistemas Digitales**
**Base de Datos I (SD1006)**

## 2.- Inventario de Tablas del Diccionario de Datos Genérico (corregido)

| Modelo Lógico - Tablas resultantes del Diccionario Genérico (corregido) | | | |
|---|---|---|---|
| **INVENTARIO DE TABLAS**<br>**Este inventario es el mismo para los 3 SGBDs: PostgreSQL, MySQL y MS SQL Server. Todos los diccionarios deben tomar la misma cantidad y los mismos nombres de esta tablas** | | | |
| # | Tabla | Descripción Tabla | Observaciones |
| 1 | usuario | datos del usuario dentro de la red social | |
| 2 | rol | rol del usuario dentro de la red social | |
| 3 | tipo_usuario | el tipo de puesto que tiene dentro de la institucion o en la red social | |
| 4 | perfil | descripción del usuario | |
| 5 | publicacion | difusión informativa sobre temas de interés universitarios | |
| 6 | comentario | opinión de los integrantes de la red pascualina | |
| 7 | grupo | interaccion de un grupo | |
| 8 | tipo_servicio | tipos de asistencias que se brindan a los usuarios | |
| 9 | servicio | asistencia en la necesidad que puedan tener los usuarios | |
| 10 | tipo_producto | interaccion de un tipo de producto | |
| 11 | producto | artículos disponibles en la plataforma | |
| 12 | tipo_evento | interaccion de un tipo de evento | |
| 13 | evento | organización de grandes reuniones que incitan el aprendizaje | |
| 14 | venta | transacción entre el usuario y un vendedor a través de la red social pascualina | |
| 15 | publicidad | apartado que promociona un evento, servicios y demás | |
| 16 | reaccion | muestra de que el comentario o la publicación fue recibida de buena o mala manera por los usuarios | |
| 17 | notificacion | aviso sobre algun mensaje de la red social pascualina | |
| 18 | mensaje | interacción privada entre los usuarios a través de la red social | |
| 19 | solicitud_servicio | interaccion de un usuario de un tipo de servicio | |
| 20 | miembro_grupo | interaccion de un usuario a un grupo | |
| 21 | inscripcion_evento | interaccion de un usuario a un evento | |
| 22 | reaccion_comentario | Cantidad de reacciones que recibe un comentario | |
| 23 | tipo_usuario_usuario | nace de la relacion usuario a un tipo de usuario | |
| 24 | publicidad_reaccion | nace de la relacion publicidad y reaccion | |

**Institución Universitaria Pascual Bravo**
**Facultad de Ingeniería - Departamento de Sistemas Digitales**
**Base de Datos I (SD1006)**

## 3. Diccionario de Datos y Scripts de PostgreSQL

### 3.1- Diccionario de Datos Físico (SGBD PostgreSQL)

**PostgreSQL - Tablas en Detalle**

| 1 | Tabla | usuario | Descripción Tabla | datos del usuario dentro de la red social | | | | Versión | 1.1 |
|---|---|---|---|---|---|---|---|---|---|
| # | Campo | Descripción Campo | Tipo Dato - PostgreSQL | Tamaño | Nulo | PK | FK | UK | Nota/Comentarios |
| 1 | id_cedula | documento con el que se puede identificar al usuario | integer | 15 | No | $ | | | |
| 2 | p_nombre | primer nombre del usuario | varchar | 50 | No | | | | |
| 3 | s_nombre | segundo nombre del usuario | varchar | 50 | Si | | | | |
| 4 | p_apellido | primer apellido del usuario | varchar | 50 | No | | | | |
| 5 | s_apellido | segundo apellido del usuario | varchar | 50 | Si | | | | |
| 6 | direccion | lugar de residencia del usuario | varchar | 100 | Si | | | | |
| 7 | correo | método de contacto para la comunicación y recibimiento de informacion y envio de | varchar | 100 | No | | | $ | |
| 8 | clave | números y caracteres que delimitan y protegen el acceso a una cuenta | varchar | 15 | No | | | | |
| 9 | nombre_usuario | nombre que verán los demás usuarios de la red social pascualina | varchar | 30 | No | | | $ | |
| 10 | id_rol | identificación por número a cada uno de los roles | integer | 6 | No | | $ | | |
| N | | | | | | | | | |

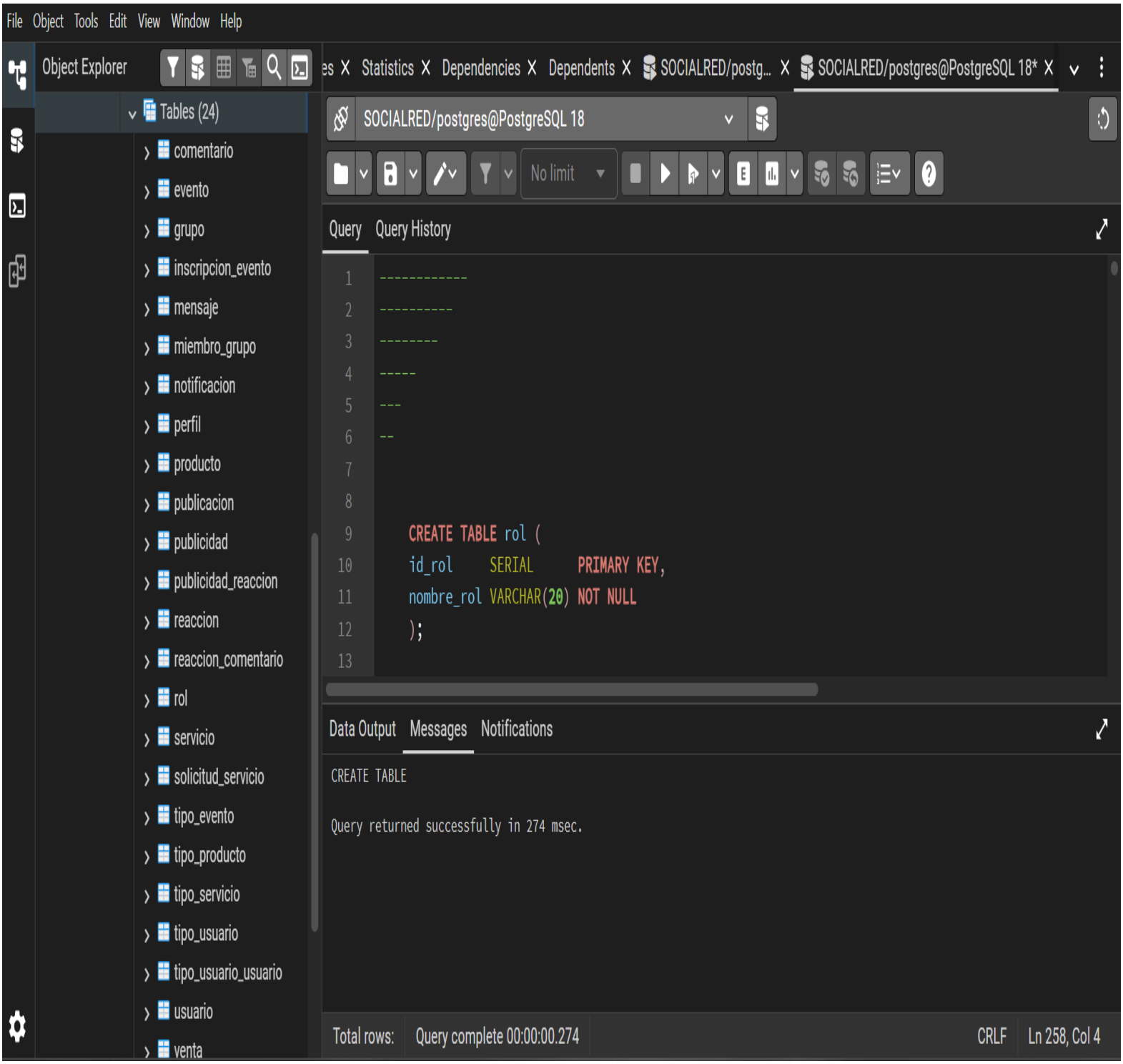
| 2 | Tabla | rol | Descripción Tabla | rol del usuario dentro de la red social | | | | Versión | 1.1 |
|---|---|---|---|---|---|---|---|---|---|
| # | Campo | Descripción Campo | Tipo Dato - PostgreSQL | Tamaño | Nulo | PK | FK | UK | Nota/Comentarios |
| 1 | id_rol | identificación por número a cada uno de los roles | serial | 6 | No | $ | | | |
| 2 | nombre_rol | nombre de los roles de la base de datos | varchar | 20 | No | | | | |

| 3 | Tabla | tipo_usuario | Descripción Tabla | el tipo de puesto que tiene dentro de la institucion o en la red social | | | | Versión | 1.1 |
|---|---|---|---|---|---|---|---|---|---|
| # | Campo | Descripción Campo | Tipo Dato - PostgreSQL | Tamaño | Nulo | PK | FK | UK | Nota/Comentarios |
| 1 | id_tipo_usuario | Identificación por número a cada uno de los tipos de usuario que puede haber | serial | 5 | No | $ | | | Autoincrementable — el sistema genera el valor automáticamente |
| 2 | n_tipo_usuario | nombre identificatorio de los tipos de usuario que cumple dentro de la institución universitaria, ya sea profesor estudiante | varchar | 20 | No | | | | |

Tipos de Dato | Invetario de Tablas | Diccionario Datos - PostgreSQL | Diccionario Datos - MySQL | Diccionario Datos - MS SQL Serv

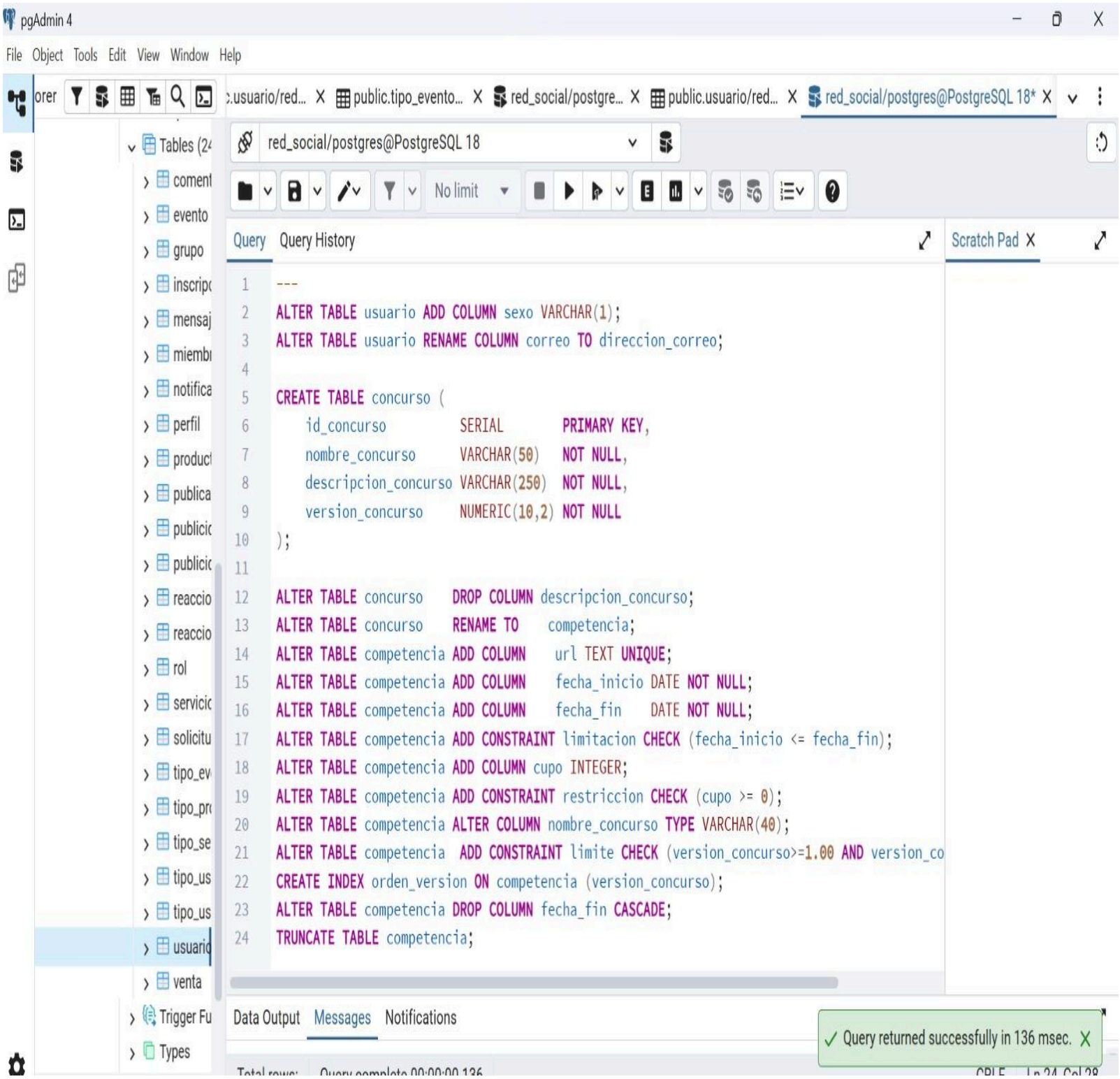
**Institución Universitaria Pascual Bravo**
**Facultad de Ingeniería - Departamento de Sistemas Digitales**
**Base de Datos I (SD1006)**

## 3.2.- Lenguaje de Definición de Dato (DDL) - Scripts de CREACIÓN de Bases de Datos



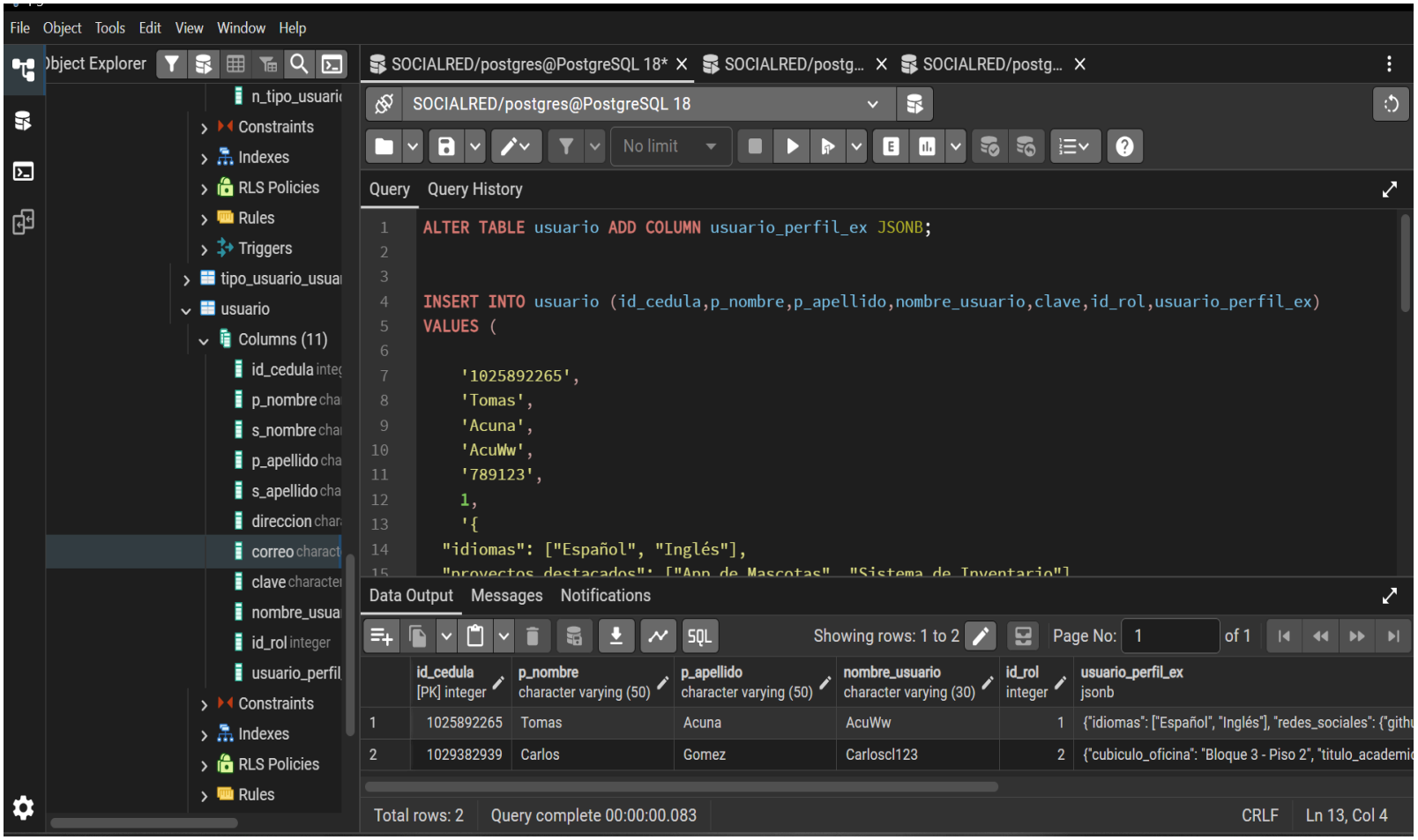
**Institución Universitaria Pascual Bravo**
**Facultad de Ingeniería - Departamento de Sistemas Digitales**
**Base de Datos I (SD1006)**

## 3.3..- Lenguaje de Definición de Dato (DDL) - Scripts de MODIFICACIÓN de Bases de Datos ESTRUCTURADOS



**Institución Universitaria Pascual Bravo**
**Facultad de Ingeniería - Departamento de Sistemas Digitales**
**Base de Datos I (SD1006)**

## 3.4..- Lenguaje de Definición de Dato (DDL) - Scripts de MODIFICACIÓN de Bases de Datos para datos SEMI ESTRUCTURADOS



## 4. Diccionario de Datos y Scripts de MySQL

### 4.1- Diccionario de Datos Físico (SGBD MySql)

K322:N322 | Tamaño

**MySQL - Tablas en Detalle**

| 1 | Tabla | usuario | Descripción Tabla | datos del usuario dentro de la red social | | | | Versión | 1.0 |
|---|---|---|---|---|---|---|---|---|---|
| # | Campo | Descripción Campo | Tipo Dato - MySQL | Tamaño | Nulo | PK | FK | UK | Nota/Comentarios |
| 1 | id_cedula | documento con el que se puede identificar al usuario | int | 15 | No | $ | | | |
| 2 | p_nombre | primer nombre del usuario | varchar | 50 | No | | | | |
| 3 | s_nombre | segundo nombre del usuario | varchar | 50 | Si | | | | |
| 4 | p_apellido | primer apellido del usuario | varchar | 50 | No | | | | |
| 5 | s_apellido | segundo apellido del usuario | varchar | 50 | Si | | | | |
| 6 | direccion | lugar de residencia del usuario | varchar | 100 | Si | | | | |
| 7 | correo | método de contacto para la comunicación y recibimiento de informacion y envio de | varchar | 100 | No | | | $ | |
| 8 | clave | números y caracteres que delimitan y protegen el acceso a una cuenta | varchar | 15 | No | | | | |
| 9 | nombre_usuario | nombre que verán los demás usuarios de la red social pascualina | varchar | 30 | No | | | $ | |
| 10 | id_rol | identificación por número a cada uno de los roles | int | 6 | No | | $ | | |

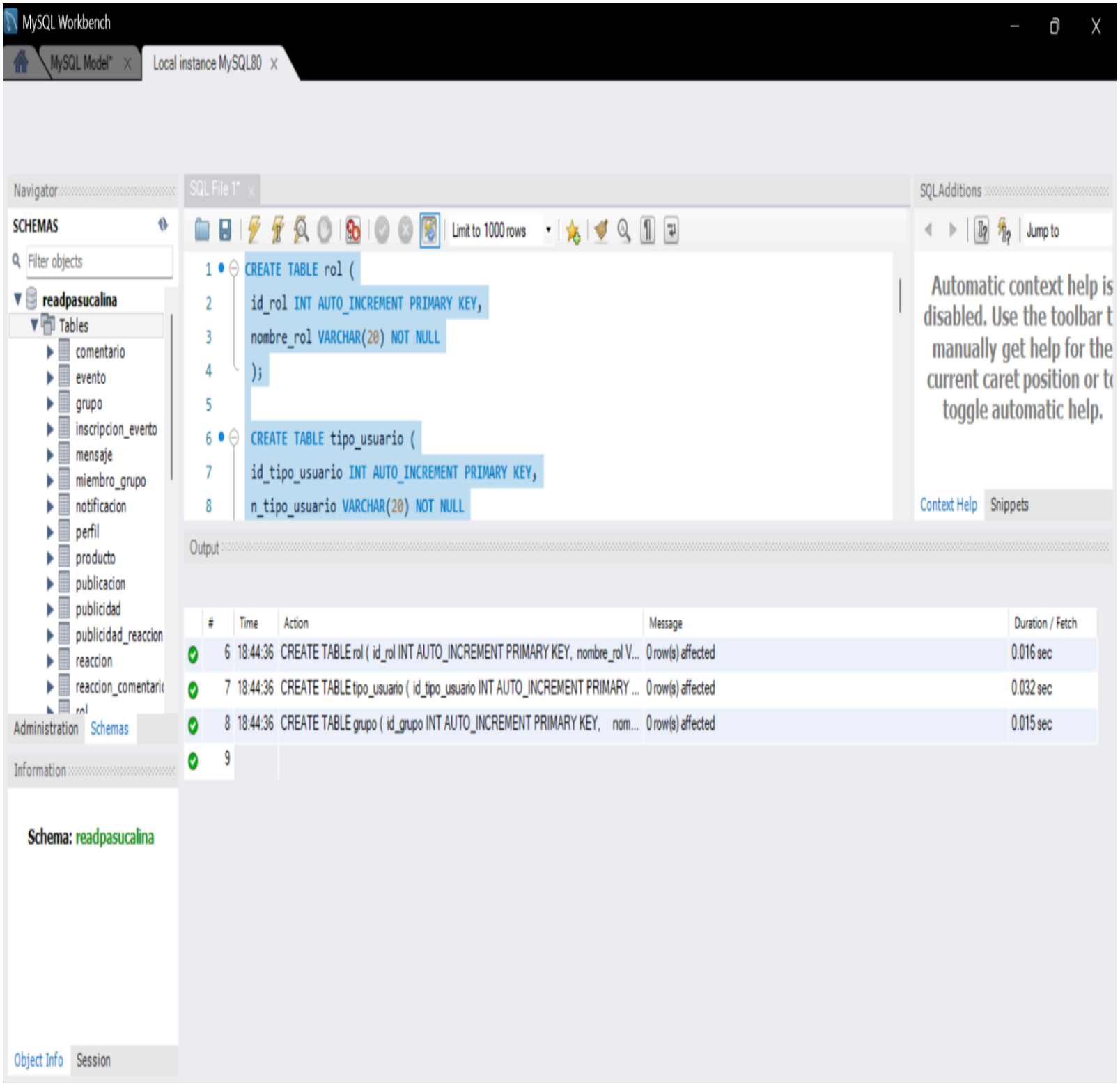
| 2 | Tabla | rol | Descripción Tabla | rol del usuario dentro de la red social | | | | Versión | 1.0 |
|---|---|---|---|---|---|---|---|---|---|
| # | Campo | Descripción Campo | Tipo Dato - MySQL | Tamaño | Nulo | PK | FK | UK | Nota/Comentarios |
| 1 | id_rol | identificación por número a cada uno de los roles | auto_increment | 6 | No | $ | | | |
| 2 | nombre_rol | nombre de los roles de la base de datos | varchar | 20 | No | | | | |

| 3 | Tabla | tipo_usuario | Descripción Tabla | el tipo de puesto que tiene dentro de la institucion o en la red social | | | | Versión | 1.0 |
|---|---|---|---|---|---|---|---|---|---|
| # | Campo | Descripción Campo | Tipo Dato - MySQL | Tamaño | Nulo | PK | FK | UK | Nota/Comentarios |
| 1 | id_tipo_usuario | Identificación por número a cada uno de los tipos de usuario que puede haber | auto_increment | 5 | No | $ | | | Autoincrementable — el sistema genera el valor automáticamente |
| 2 | n_tipo_usuario | nombre identificatorio de los tipos de usuario que cumple dentro de la institución universitaria, | varchar | 20 | No | | | | |

Tipos de Dato | Inventario de Tablas | Diccionario Datos - PostgreSQL | Diccionario Datos - MySQL | Diccionario Datos - MS SQL Serv

**Institución Universitaria Pascual Bravo**
**Facultad de Ingeniería - Departamento de Sistemas Digitales**
**Base de Datos I (SD1006)**

## 4.2.- MYSQL. Lenguaje de Definición de Dato (DDL) - Scripts de CREACIÓN de Bases de Datos



**Institución Universitaria Pascual Bravo**
**Facultad de Ingeniería - Departamento de Sistemas Digitales**
**Base de Datos I (SD1006)**

## 5. Diccionario de Datos y Scripts de MS SQL Server

### 5.1- Diccionario de Datos Físico (SGBD MS SQL Server)

**MS SQL Server - Tablas en Detalle**

| 1 | Tabla | usuario | Descripción Tabla | datos del usuario dentro de la red social | | | | Versión | 1.0 |
|---|---|---|---|---|---|---|---|---|---|
| # | Campo | Descripción Campo | Tipo Dato - MS SQL Server | Tamaño | Nulo | PK | FK | UK | Nota/Comentarios |
| 1 | id_cedula | documento con el que se puede identificar al usuario | integer | 15 | No | $ | | | |
| 2 | p_nombre | primer nombre del usuario | varchar | 50 | No | | | | |
| 3 | s_nombre | segundo nombre del usuario | varchar | 50 | Si | | | | |
| 4 | p_apellido | primer apellido del usuario | varchar | 50 | No | | | | |
| 5 | s_apellido | segundo apellido del usuario | varchar | 50 | Si | | | | |
| 6 | direccion | lugar de residencia del usuario | varchar | 100 | Si | | | | |
| 7 | correo | método de contacto para la comunicación y recibimiento de informacion y envio de | varchar | 100 | No | | | $ | |
| 8 | clave | números y caracteres que delimitan y protegen el acceso a una cuenta | varchar | 15 | No | | | | |
| 9 | nombre_usuario | nombre que verán los demás usuarios de la red social pascualina | varchar | 30 | No | | | $ | |
| 10 | id_rol | identificación por número a cada uno de los roles | integer | 6 | No | | $ | | |

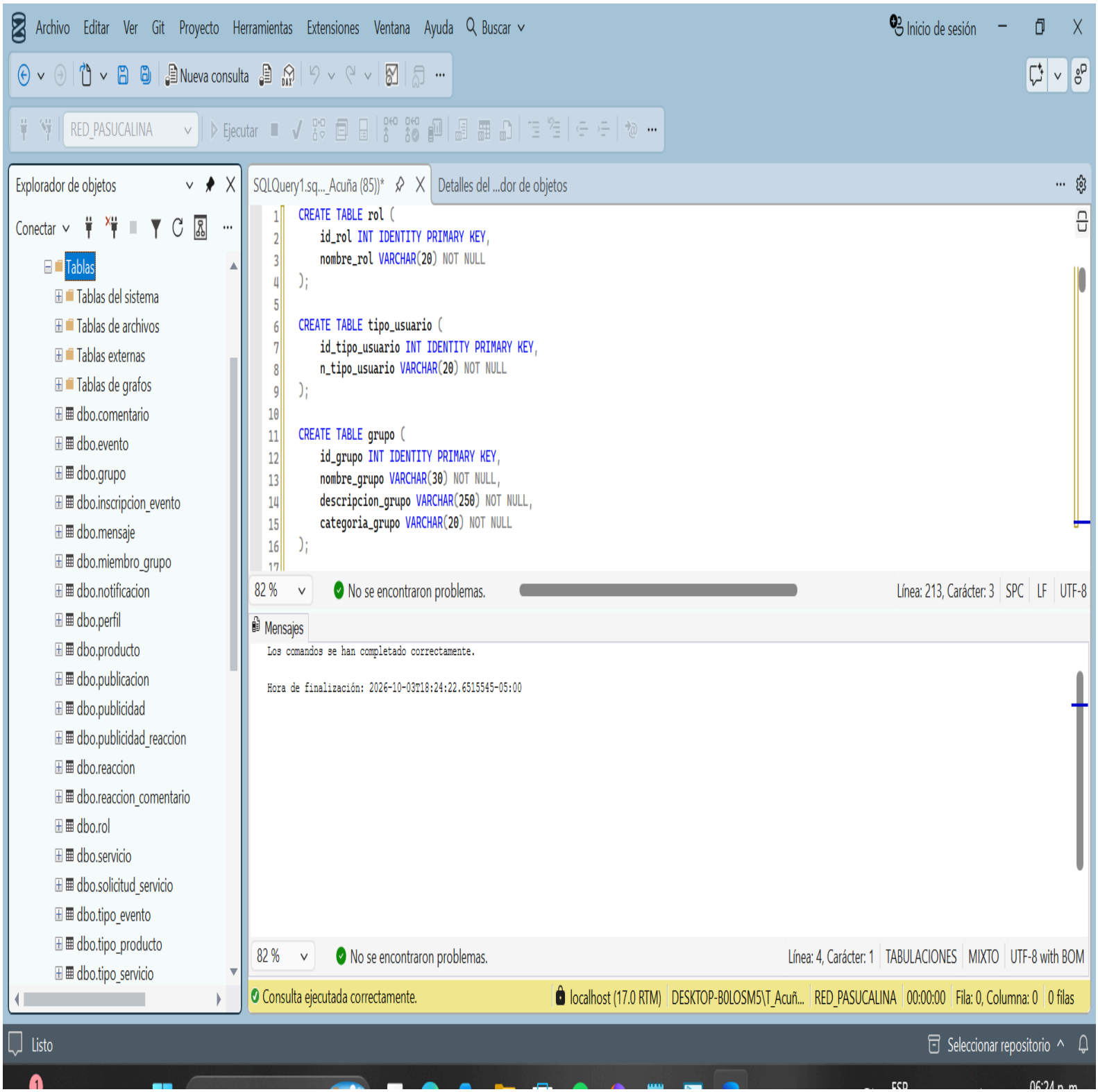
| 2 | Tabla | rol | Descripción Tabla | rol del usuario dentro de la red social | | | | Versión | 1.0 |
|---|---|---|---|---|---|---|---|---|---|
| # | Campo | Descripción Campo | Tipo Dato - MS SQL Server | Tamaño | Nulo | PK | FK | UK | Nota/Comentarios |
| 1 | id_rol | identificación por número a cada uno de los roles | identity | 6 | No | $ | | | |
| 2 | nombre_rol | nombre de los roles de la base de datos | varchar | 20 | No | | | | |

| 3 | Tabla | tipo_usuario | Descripción Tabla | el tipo de puesto que tiene dentro de la institucion o en la red social | | | | Versión | 1.0 |
|---|---|---|---|---|---|---|---|---|---|
| # | Campo | Descripción Campo | Tipo Dato - MS SQL Server | Tamaño | Nulo | PK | FK | UK | Nota/Comentarios |
| 1 | id_tipo_usuario | Identificación por número a cada uno de los tipos de usuario que puede haber | identity | 5 | No | $ | | | Autoincrementable — el sistema genera el valor automáticamente |
| 2 | n_tipo_usuario | nombre identificatorio de los tipos de usuario que cumple dentro de la institución | varchar | 20 | No | | | | |

Tipos de Dato | Inventario de Tablas | Diccionario Datos - PostgreSQL | Diccionario Datos - MySQL | Diccionario Datos - MS SQL Serv

**Institución Universitaria Pascual Bravo**
**Facultad de Ingeniería - Departamento de Sistemas Digitales**
**Base de Datos I (SD1006)**

## 5.2.- SQL Server. Lenguaje de Definición de Dato (DDL) - Scripts de CREACIÓN de Bases de Datos



**6.- Análisis de resultados (Grupo)**

Al revisar nuestro trabajo realizado durante el diseño de nuestra base de datos de la red social pascualina, como equipo pudimos verificar que la estructura planteada no solo responde a los requerimientos del sistema, sino que también garantiza un manejo eficiente, ordenado y seguro de la información y uno de los puntos clave en este proceso fue el análisis y selección de los tipos de datos para los distintos gestores como lo son PostgreSQL, MySQL y MS SQL Server y nos dimos cuenta que de lo importante que es no elegir tipos de datos al azar o a lo loco, nosotros tuvimos que definir con precisión qué campos requerían números exactos como (DECIMAL/NUMERIC) y cuáles se beneficiaban de un identificador único (UUID) y cómo manejar la información temporal (DATE o TIMESTAMP) que nos permitió cuidar la precisión de los datos y optimizar el uso de memoria y además, el uso de formatos modernos como JSONB o campos binarios para archivos multimedia demostró que el modelo está preparado para escalar sin volverse lento y respecto a las relaciones entre las tablas, tambien logramos estructurar una arquitectura sólida en la que cada entidad se conecta mediante llaves primarias y foráneas bien definidas y esto evitó la aparición de datos duplicados o registros "huérfanos" como nos explico el profesor en la clase de bases de datos, garantizando que cualquier operación de inserción, actualización o borrado mantenga la integridad de toda la base de datos y por último, al comparar el avance desde el Modelo Conceptual hasta el físico, confirmamos que existió una continuidad lógica y coherente en todo momento y el Modelo Conceptual nos sirvió para entender el negocio y sus reglas sin preocuparnos por la parte técnica y en el Modelo Lógico organizamos esa información en tablas normalizadas (alcanzando la 3FN) y resolvimos las relaciones complejas ($M$:$N$) mediante tablas intermedias y en el Modelo Físico traducimos todo eso al lenguaje ejecutable (DDL) de cada motor de base de datos, respetando sus sintaxis y restricciones particulares y en conclusión, este análisis nos demuestra que el modelo final es robusto, coherente y funcional, listo para implementarse en un entorno real con un gran equilibrio entre rendimiento e integridad de la información.

**7.- Reflexiones y conclusiones individuales**

**Samuel Perez Jaramillo:**

Yo hablando desde mi punto de vista o perspectiva como dicen algunos, para mi la creación de los modelos de datos y la ejecución de los scripts DDL fue un proceso sumamente inmersivo, especialmente al llevar la estructura conceptual a la práctica real. Lo valioso de ponerse a prueba es que se aprende mucho más y se gana experiencia directa. Me quedó totalmente claro cómo manejar la integridad referencial, cuándo implementar identificadores autoincrementables como SERIAL o AUTO_INCREMENT para llaves primarias y cuándo utilizar tipos numéricos para las claves foráneas. Asimismo, hubo detalles técnicos y delicados de sintaxis a los que tocó prestar atención, como comprender que tipos como DATE no llevan longitud o que definir bien la precisión en NUMERIC(p,s) que previene desbordamientos y cosas tan minuciosas como comas sobrantes o nombres duplicados

en los CONSTRAINT nos obligaron a revisar el código con lupa a nivel académico y profesional, este aprendizaje cambió sustancialmente mi perspectiva sobre la arquitectura de software en las bases de datos y ahora entiendo que un esquema relacional bien diseñado y normalizado es la columna vertebral indispensable para que cualquier aplicación sea estable y segura y aplicar con criterio reglas de propagación como ON DELETE RESTRICT y ON UPDATE CASCADE me brinda herramientas reales para prevenir registros huérfanos, lo cual me fortalece para avanzar en la carrera de desarrollo de software y afrontar retos en el área Backend por el lado del trabajo en equipo, fue una experiencia clave y debatir con mis compañeros y recibir retroalimentación nos permitió sacar el proyecto adelante. Lo más delicado fue coordinar la estructura para evitar redundancias, unificar la nomenclatura de restricciones y revisar el orden estricto de creación de tablas para evitar fallos de dependencias. Esa revisión colaborativa nos ayudó a corregir errores a tiempo y finalmente, construir este modelo para el caso de estudio me dio una visión clara de cómo estructurar interacciones en una plataforma real. Garantizar un diseño normalizado e íntegro asegura que el sistema pueda crecer a futuro de forma ágil, demostrando que el modelado de datos es la base sólida de cualquier solución informática.

**Tomas Acuña Alzate:**

Al reflexionar sobre el desarrollo de esta actividad, considero que trabajar en la transición desde los requerimientos iniciales hasta la implementación del modelo físico representó un aprendizaje profundamente enriquecedor. Llevar la teoría de la materia a la práctica mediante la comparación de motores como PostgreSQL, MySQL y SQL Server me permitió comprender que el rendimiento de una base de datos depende directamente del rigor con el que se seleccionan los tipos de datos y se delimita su dominio. Aprendí a valorar los detalles finos del lenguaje DDL, entendiendo cómo influyen la asignación en bytes de cada columna, la diferencia entre guardar cadenas de longitud fija o variable, y la precisión requerida en tipos decimales y temporales para evitar la pérdida o corrupción de información crítica en la persistencia del sistema y en el ámbito académico y en mi proyección profesional dentro del desarrollo de software, este proyecto marca un punto de inflexión. Entendí que la lógica de programación no vive aislada en el código fuente, sino que depende directamente de una arquitectura de datos sólida. Saber estructurar restricciones de integridad, seleccionar entre un patrón UUID o una clave identidad, y normalizar un esquema hasta la tercera forma normal (3FN) me da la solvencia técnica para proponer e implementar soluciones backend de calidad profesional que optimicen el uso de memoria y reduzcan la latencia en las consultas y la experiencia de trabajo colaborativo resultó ser un pilar fundamental durante toda la actividad. Coordinar las entregas entre varios integrantes exigió una comunicación permanente y transparente para alinear conceptos. Fue especialmente valioso el proceso de revisión mutua, donde debatimos la pertinencia de ciertos atributos, corregimos nombres de restricciones para mantener un estándar limpio y reorganizamos relaciones interconectadas. Aprender a escuchar otros puntos de vista, argumentar decisiones de diseño y ajustar nuestra propuesta en función del consenso fortaleció notablemente nuestras habilidades de trabajo en equipo y por último, el diseño del modelo de datos para el caso de estudio propuesto demostró la importancia crítica de contar con una estructura relacional robusta. El caso de

estudio nos enfrentó al reto de abstraer procesos del mundo real y traducirlos en tablas, cardinalidades y restricciones estrictas (CHECK, UNIQUE, FOREIGN KEY). Lograr que las tablas asociativas mantengan la unicidad sin generar campos autoincrementables innecesarios garantiza que la información se conserve consistente a lo largo del tiempo, consolidando una base de datos totalmente preparada para escalar de forma confiable ante el crecimiento del sistema.

**Samuel Moreno Salazar:**

A lo largo de la realización de este informe y la elaboración de los esquemas relacionales, logré consolidar una visión mucho más madura sobre la importancia del diseño de bases de datos. Comprender de manera detallada el comportamiento del dominio en los tipos de datos, así como el impacto que tiene cada byte reservado en el almacenamiento general, me enseñó que la eficiencia de un sistema se planifica desde la fase de modelado. Experimentar con la creación de tablas, la definición de llaves primarias compuestas en tablas intermedias y la configuración de llaves foráneas me permitió afianzar conceptos de integridad referencial que antes resultaban puramente teóricos, llevándolos a un terreno práctico y ejecutable desde el punto de vista académico y profesional, este trabajo aportó herramientas conceptuales y técnicas de gran valor para mi formación en tecnología de software. Hoy reconozco que la estabilidad y seguridad de cualquier aplicación web o móvil dependen de la calidad de su modelo de datos subyacente. Entender cómo las reglas de propagación de cambios evitan la existencia de datos huérfanos y de qué manera la correcta normalización elimina la redundancia me prepara directamente para asumir proyectos de mayor complejidad técnica en el entorno laboral, permitiéndome diseñar arquitecturas que soporten un alto volumen de transacciones de forma ágil el trabajo en equipo desempeñó un rol crucial en el logro de los objetivos planteados. El intercambio continuo de ideas y la retroalimentación entre compañeros nos obligaron a mantener un ritmo de trabajo coordinado y riguroso. Un reto importante fue unificar la sintaxis y los nombres de los objetos dentro del script para evitar fallos de compilación o conflictos de dependencias entre entidades relacionadas. Realizar revisiones cruzadas de código no solo sirvió para detectar comas sobrantes o tipos de datos incompatibles en las claves foráneas, sino que también promovió un aprendizaje dinámico y colaborativo entre todos los miembros del grupo en conclusión, la elaboración del modelo de datos para el caso de estudio permitió transformar necesidades operativas en una arquitectura de software limpia y estructurada. El caso de estudio exigió abstraer entidades, delimitar roles y asegurar que cada transacción respete las reglas de negocio establecidas. Este proceso demostró que una base de datos bien diseñada no solo previene fallas de consistencia a futuro, sino que representa el cimiento indispensable sobre el cual se construyen soluciones tecnológicas duraderas, eficientes y verdaderamente profesionales

**Santiago Mejia Torres:**

Desde mi punto de vista, participar en el diseño y la implementación del script DDL para este caso de estudio representó un ejercicio de aprendizaje dinámico y profundamente enriquecedor. Llevar la teoría del modelado conceptual hacia un esquema físico en motores como PostgreSQL, MySQL y SQL

Server me permitió comprender el impacto directo que tiene la elección de cada tipo de dato sobre la eficiencia de un sistema. Entendí la importancia de controlar los dominios de la información, aplicando tipos numéricos precisos, formateando adecuadamente los atributos temporales y configurando llaves primarias autoincrementables como SERIAL o claves compuestas según las necesidades de cada tabla. Asimismo, enfrentarnos a errores de sintaxis, como la asignación incorrecta de longitudes o conflictos en las restricciones CONSTRAINT, nos enseñó a ser mucho más rigurosos al escribir y auditar código DDL en el plano académico y profesional, esta experiencia transformó por completo mi entendimiento sobre la ingeniería de datos. Comprobé que el éxito de una aplicación informática no radica únicamente en la interfaz o la lógica de negocio, sino en la solidez de su base de datos. Saber normalizar una estructura hasta la tercera forma normal (3FN) y configurar reglas de integridad referencial como ON DELETE RESTRICT me brinda competencias clave para evitar la redundancia y asegurar la persistencia limpia de los datos, algo indispensable para desempeñarme con éxito en el desarrollo backend y en cuanto al trabajo en equipo, la colaboración fue un factor determinante para lograr un resultado cohesionado. Integrar distintas perspectivas al momento de definir cardinalidades y revisar la sintaxis exigió una comunicación clara y continua. El proceso de revisión cruzada nos permitió detectar dependencias circulares entre llaves foráneas y corregir el orden de creación de las tablas a tiempo por último, desarrollar el modelo de datos para el caso de estudio demostró que una planificación cuidadosa garantiza la escalabilidad del software. Una base de datos bien estructurada y sin inconsistencias es el cimiento necesario para que cualquier sistema responda de manera rápida y segura ante un crecimiento constante.

## 8.- Informe (calidad de presentación y respeto de los requerimientos)

## 9.- Video de Sustentación

https://drive.google.com/file/d/1uWrKNzZ0InjY7CvvxKXYTo-AMn26W24T/view?usp=sharing

## 10.- Repositorio GIT



# ANEXO B

## Ejemplo de Inserción/Consulta de Información No Relacional
## Tipos de Datos Semi-Estructurados (JSON/JSONB)

```
- Datos para un estudiante
INSERT INTO usuarios (nombre, correo, tipo_usuario, perfil_extendido)
VALUES (
    'Ana Torres',
    'ana.torres@pascual.edu.co',
    'estudiante',
    '{
        "intereses": ["programación", "videojuegos", "IA"],
        "habilidades": [
            {"nombre": "Python", "nivel": "intermedio"},
            {"nombre": "SQL", "nivel": "básico"}
        ],
        "materias_actuales": ["Bases de Datos I", "Algoritmos"],
        "disponibilidad": {
            "dias": ["lunes", "miércoles"],
            "horario": "tarde"
        }
    }'
);

- Datos para un docente
INSERT INTO usuarios (nombre, correo, tipo_usuario, perfil_extendido)
VALUES (
    'Carlos Gómez',
    'carlos.gomez@pascual.edu.co',
    'docente',
    '{
        "lineas_investigacion": ["Big Data", "IoT"],
        "publicaciones": [
            {"titulo": "Análisis de Datos", "anio": 2023},
            {"titulo": "IoT en educación", "anio": 2024}
        ],
        "disponibilidad_tutoria": {
            "dias": ["martes", "jueves"],
            "horario": "mañana"
        }
    }'
);

- Consultas
- Buscar usuarios por perfil
SELECT id_usuario,nombre,tipo_usuario FROM usuarios WHERE tipo_usuario= 'estudiante';

- Buscar usuarios interesados en "IA"
SELECT nombre, perfil_extendido FROM usuarios
WHERE perfil_extendido -> 'intereses' ? 'IA';

- Obtener habilidades de un usuario
SELECT nombre, perfil_extendido -> 'habilidades' AS habilidades FROM usuarios;
```

**Institución Universitaria Pascual Bravo**
**Facultad de Ingeniería - Departamento de Sistemas Digitales**
**Base de Datos I (SD1006)**